SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning,*
and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 4 **Total: Marks:40**

Annexure A

5

Code of Conduct:

I A G Jayapradhan (192471016) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Annexure A

INDUSTRY PROBLEM TASK

INDUSTRY SCENARIO

Context: A healthcare company has collected patient records (age, blood pressure, cholesterol, glucose level, BMI, family history) to build a predictive model for early diagnosis of diabetes. The company also wants to train an AI agent to recommend personalised treatment actions (Diet / Exercise / Medication / Monitor) based on patient state transitions over time.

You are hired as an AI/ML Engineer. Address the following tasks:

Task 1: Data Preparation & Inductive Learning

(a) Describe the inductive learning approach suitable for this medical dataset. Identify the target variable and at least three key features with justification.

(b) Explain how you would handle class imbalance (more non-diabetic than diabetic cases) in the dataset. Suggest and justify a suitable balancing strategy (SMOTE / under-sampling / class weights).

(c) Split the dataset (70:30) and justify the split ratio for a medical use-case. State the preprocessing steps (normalisation, encoding) applied and their impact.

Task 2: Decision Tree for Diagnosis

(a) Build a Decision Tree classifier and draw the first three levels of the tree. Justify the root node selection using Information Gain / Gini Index values.

(b) Evaluate the model: report Accuracy, Precision, Recall, and F1-Score. Identify False Negatives (missed diabetes cases) and suggest how to reduce them—justify clinically.

(c) Apply post-pruning (cost-complexity pruning). Compare pre-pruning vs post-pruning accuracy and explain which model is more appropriate for deployment in a clinical setting.

Task 3: Statistical Learning for Risk Stratification

(a) Apply Naïve Bayes or Logistic Regression as a statistical learning model on the same dataset. Compare its performance (Accuracy, AUC-ROC) with the Decision Tree. Discuss when a statistical model is preferable.

(b) Perform feature importance analysis. Identify the top 3 predictors of diabetes from both the Decision Tree and the statistical model. Do they agree? Justify your findings.

Task 4: Reinforcement Learning for Treatment Recommendation

(a) Model the treatment recommendation as an MDP. Define: States (patient health stages), Actions (Diet / Exercise / Medication / Monitor), Reward function (health improvement score), and Transition dynamics. Justify each component.

(b) Apply Q-Learning to train the agent for at least 5 patient episodes. Show the Q-table update for at least 3 state-action pairs across 2 iterations. State the convergence criterion used.

(c) Extract the final learned policy. Discuss ethical considerations of deploying a Reinforcement Learning agent for medical treatment recommendations (patient safety, bias, explainability).

INDUSTRY PROBLEM TASK – ASSESSMENT

Healthcare AI/ML: Diabetes Diagnosis and Treatment Recommendation

Industry Scenario

A healthcare company has collected patient records containing age, blood pressure, cholesterol, glucose, BMI, family history and lifestyle information. The objective is to build an AI system for early diabetes diagnosis and to recommend personalised treatment actions based on patient-state transitions over time. This assessment addresses data preparation, supervised learning, statistical learning, and reinforcement learning.

Dataset Used

For demonstration, the program uses scikit-learn's built-in Diabetes dataset and converts the continuous disease progression target into a binary risk label using its median value. In a real healthcare deployment, the same workflow should be applied to a clinically validated patient dataset containing the variables specified in the scenario.

Task 1 – Data Preparation & Inductive Learning

1(a) Data Preparation

Missing values should first be identified using isnull().sum(). Numerical variables can be imputed using the median because it is less sensitive to outliers. Categorical variables, if present, should be encoded using one-hot encoding. Features should be separated from the target before model training.

Target variable: Diabetes Risk (0 = lower risk, 1 = higher risk).

Features: age, sex, BMI, blood pressure and serum-related measurements available in the demonstration dataset.

1(b) Class Imbalance

If non-diabetic cases greatly outnumber diabetic cases, accuracy alone can be misleading. Suitable approaches include SMOTE applied only to the training data, controlled under-sampling, and class_weight='balanced'. The chosen method should be evaluated using precision, recall, F1-score and ROC-AUC. For medical screening, recall/sensitivity is particularly important because missing a high-risk patient can have greater consequences than producing an additional false positive.

1(c) 70:30 Split and Preprocessing

A 70:30 train-test split is appropriate for this assessment because most records remain available for learning while a separate 30% provides a meaningful estimate of generalisation. Stratification should be used to preserve the class ratio. Scaling is applied inside a pipeline for Logistic Regression and Naive Bayes when appropriate. The test set must remain untouched until final evaluation to prevent data leakage.

Task 2 – Decision Tree for Diagnosis

2(a) Model Construction

A Decision Tree classifier is trained using entropy/information gain. The root is selected as the feature producing the largest reduction in entropy. The first three levels can be displayed with sklearn's plot_tree(). Limiting max_depth for visualisation makes the structure easier to interpret.

Root-node justification: the feature at the root is selected because it provides the highest information gain among the available features for the training data.

2(b) Model Evaluation

The model is evaluated using Accuracy, Precision, Recall and F1-score. The confusion matrix identifies true positives, true negatives, false positives and false negatives. False negatives (patients predicted as non-diabetic although they are high-risk) must be examined carefully because they may delay clinical attention.

2(c) Pre-pruning vs Post-pruning

Pre-pruning restricts tree growth during training using parameters such as max_depth, min_samples_split and min_samples_leaf. Post-pruning first grows a larger tree and then removes weak branches using cost-complexity pruning (ccp_alpha). Post-pruning is often useful when interpretability and generalisation are important because unnecessary branches can be removed after the tree has captured the training structure.

Task 3 – Statistical Learning for Risk Stratification

3(a) Naive Bayes / Logistic Regression

Logistic Regression models the probability of the positive diabetes-risk class and is highly interpretable through its coefficients. Naive Bayes is a probabilistic classifier based on conditional independence assumptions. Both can be compared with the Decision Tree using Accuracy and ROC-AUC. Logistic Regression is generally preferable when a transparent linear relationship and probability-based risk score are desired; a Decision Tree is preferable when readable if-then rules and non-linear splits are valuable.

3(b) Feature Importance

Decision Tree importance is based on the reduction in impurity contributed by each feature. Logistic Regression importance can be examined through the absolute value of standardised coefficients. The two rankings need not be identical because the models learn different relationships. Agreement among strong predictors increases confidence, but clinical importance should ultimately be validated by domain experts rather than inferred only from model statistics.

Task 4 – Reinforcement Learning for Treatment Recommendation

4(a) MDP Formulation

States represent patient health stages such as Stable, At-Risk, Uncontrolled, and Improved. Actions include Diet, Exercise, Medication, and Monitor. The reward represents health improvement, while treatment safety and undesirable transitions receive penalties. The transition function describes how an action changes the patient's health state. In a real clinical system, transitions must be learned from validated longitudinal data and constrained by clinical rules.

4(b) Q-Learning

Q-Learning updates the value of taking an action in a state using: $Q(s,a) = Q(s,a) + \alpha[r + \gamma \max Q(s',a') - Q(s,a)]$. At least five episodes are used to demonstrate learning. A Q-table is recorded at selected episodes to show how action values evolve.

4(c) Final Policy and Ethics

The final policy selects the action with the largest Q-value for each state. A healthcare reinforcement-learning agent must not be allowed to autonomously prescribe or change medication without clinical oversight. Important considerations include patient safety, bias across demographic groups, privacy, informed consent, explainability, auditability, uncertainty handling and a human-in-the-loop clinical approval process.

Industry Problem Task – Assessment

Complete Python Implementation

```
# INDUSTRY PROBLEM TASK
# Diabetes diagnosis + risk stratification + Q-learning demo

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix, roc_auc_score,
classification_report)
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.inspection import permutation_importance

# ------------------------------------------------------------
# TASK 1: DATA PREPARATION
# ------------------------------------------------------------
data = load_diabetes(as_frame=True)
df = data.frame.copy()

# Convert continuous target into binary risk using the median.
df["Diabetes_Risk"] = (df["target"] >= df["target"].median()).astype(int)

X = df.drop(columns=["target", "Diabetes_Risk"])
y = df["Diabetes_Risk"]

print("First 5 rows:")
print(df.head())
print("\nData types:")
print(df.dtypes)
print("\nMissing values:")
print(df.isnull().sum())

X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.30, random_state=42, stratify=y
)

# ------------------------------------------------------------
# TASK 2: DECISION TREE
# ------------------------------------------------------------
tree = DecisionTreeClassifier(
```

```
criterion="entropy",
random_state=42,
max_depth=4
)
tree.fit(X_train, y_train)

pred_tree = tree.predict(X_test)
prob_tree = tree.predict_proba(X_test)[:, 1]

print("\nDecision Tree Results")
print("Accuracy :", accuracy_score(y_test, pred_tree))
print("Precision:", precision_score(y_test, pred_tree))
print("Recall :", recall_score(y_test, pred_tree))
print("F1-score :", f1_score(y_test, pred_tree))
print("ROC-AUC :", roc_auc_score(y_test, prob_tree))
print("Confusion Matrix:\n", confusion_matrix(y_test, pred_tree))

plt.figure(figsize=(16, 9))
plot_tree(tree, feature_names=X.columns, class_names=["Low", "High"],
max_depth=2, filled=False, rounded=True)
plt.title("First Three Levels of Decision Tree")
plt.show()

# Cost-complexity post-pruning
path = tree.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas = path.ccp_alphas
alpha = ccp_alphas[len(ccp_alphas)//2] if len(ccp_alphas) > 2 else 0.0

pruned_tree = DecisionTreeClassifier(
criterion="entropy", random_state=42, ccp_alpha=alpha
)
pruned_tree.fit(X_train, y_train)
pred_pruned = pruned_tree.predict(X_test)

print("\nPost-pruned Tree")
print("ccp_alpha:", alpha)
print("Accuracy :", accuracy_score(y_test, pred_pruned))
print("Precision:", precision_score(y_test, pred_pruned))
print("Recall :", recall_score(y_test, pred_pruned))
print("F1-score :", f1_score(y_test, pred_pruned))

# ------------------------------------------------------------
# TASK 3: LOGISTIC REGRESSION AND NAIVE BAYES
# ------------------------------------------------------------
logistic = Pipeline([
("scaler", StandardScaler()),
("model", LogisticRegression(max_iter=2000))
])
logistic.fit(X_train, y_train)

nb = GaussianNB()
nb.fit(X_train, y_train)

models = {
"Decision Tree": (tree, pred_tree, prob_tree),
"Logistic Regression": (
logistic,
logistic.predict(X_test),
logistic.predict_proba(X_test)[:, 1]
),
"Naive Bayes": (
nb,
```

```
nb.predict(X_test),
nb.predict_proba(X_test)[:, 1]
)
}

print("\nModel Comparison")
for name, (model, pred, prob) in models.items():
print(name,
"Accuracy =", round(accuracy_score(y_test, pred), 4),
"ROC-AUC =", round(roc_auc_score(y_test, prob), 4))

# Feature importance: Decision Tree
importance = pd.Series(tree.feature_importances_, index=X.columns)
print("\nDecision Tree Feature Importance:")
print(importance.sort_values(ascending=False))

# Logistic Regression coefficient importance
coef = pd.Series(
np.abs(logistic.named_steps["model"].coef_[0]),
index=X.columns
)
print("\nLogistic Regression Absolute Coefficients:")
print(coef.sort_values(ascending=False))

# ------------------------------------------------------------
# TASK 4: Q-LEARNING FOR TREATMENT RECOMMENDATION
# ------------------------------------------------------------
states = ["Stable", "At-Risk", "Uncontrolled", "Improved"]
actions = ["Diet", "Exercise", "Medication", "Monitor"]

Q = np.zeros((len(states), len(actions)))
alpha = 0.1
gamma = 0.9
epsilon = 0.2

# Example transition/reward function for an academic simulation.
# It is NOT a clinical treatment recommendation.
def environment(state, action):
if state == 0: # Stable
transitions = {
0: (3, 3), 1: (3, 3), 2: (1, -1), 3: (0, 1)
}
elif state == 1: # At-Risk
transitions = {
0: (3, 5), 1: (3, 4), 2: (3, 2), 3: (1, 1)
}
elif state == 2: # Uncontrolled
transitions = {
0: (1, 2), 1: (1, 2), 2: (3, 6), 3: (2, 0)
}
else: # Improved
transitions = {
0: (3, 2), 1: (3, 2), 2: (3, -1), 3: (0, 3)
}
return transitions[action]

episode_history = []

for episode in range(1, 6):
state = 1 # start at At-Risk
total_reward = 0
```

```
for step in range(10):
if np.random.rand() < epsilon:
action = np.random.randint(len(actions))
else:
action = np.argmax(Q[state])

next_state, reward = environment(state, action)

Q[state, action] = Q[state, action] + alpha * (
reward + gamma * np.max(Q[next_state]) - Q[state, action]
)

total_reward += reward
state = next_state

episode_history.append(total_reward)

print(f"\nEpisode {episode}")
print(np.round(Q, 2))

print("\nFinal Learned Policy")
for i, state in enumerate(states):
best_action = actions[np.argmax(Q[i])]
print(state, "->", best_action)

plt.plot(range(1, 6), episode_history, marker="o")
plt.xlabel("Episode")
plt.ylabel("Total Reward")
plt.title("Q-Learning Reward per Episode")
plt.grid()
plt.show()
```

Expected Assessment Discussion

Data preprocessing prevents missing values, inconsistent scales and categorical representations from reducing model reliability.

Accuracy should not be interpreted alone when classes are imbalanced; recall, F1-score and ROC-AUC provide additional evidence.

Decision Trees provide highly interpretable if-then rules and can model non-linear relationships, but excessive depth can cause overfitting.

Logistic Regression provides an interpretable probability-based baseline, while Naive Bayes is computationally simple but relies on distributional assumptions.

Feature-importance rankings can differ across models because each algorithm measures predictive contribution differently.

Q-Learning is suitable for sequential decision-making in an MDP, but a medical treatment agent must use safe constraints and clinician supervision.

The Q-learning treatment environment in this assessment is an educational simulation and must not be used to make real medical treatment decisions.

Conclusion

This industry problem demonstrates an end-to-end AI/ML workflow for healthcare. Data preparation establishes reliable inputs; Decision Trees provide interpretable diabetes-risk classification; Logistic Regression and Naive Bayes provide statistical-learning baselines; and Q-Learning demonstrates sequential treatment-policy learning. In a real healthcare deployment, model validation, clinical review, privacy protection, fairness testing, explainability and human oversight are essential before any system is used with patients.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation